OEIS Program Synthesis, Verified Semantics, and Graded Similarity: A Unified Account

Oruži Zarathustra Goertzel

August 14, 2026

Abstract

We collect an ongoing line of work built around the On-Line Encyclopedia of Integer Sequences (OEIS) as a testbed for verified program synthesis. Three strands meet here. First, a small domain-specific language (DSL) of integer-sequence programs is given a machine-checked operational semantics in Lean, together with a token *recognizer*, a big-step generative-semantics (GSLT) layer with modal emission specifications, and a family of formal *obligations* that a synthesizer must respect. Second, a neural sequence-to-graph synthesizer is organized so that the network's operational contracts—a legality mask, a canonical form, relabeling invariance—have machine-checked formal counterparts. Third, a graded similarity layer expressed in probabilistic-logic truth values $\langle \text{strength}, \text{confidence} \rangle$ recovers structure that an exact deduplication hash discards: it audits train/test leakage, while a conservative abstract interpreter proposes totality, sign, and parity invariants for each program. We report concrete measurements over a corpus of 104,083 synthesized programs, distinguish empirical validation of that interpreter from kernel proof, and show how verified properties could safely guide search and generate proof obligations. Finally, we give full-domain Lean correctness proofs for thirty-three frozen discoveries from our campaign and four previously reported programs: the Hofstadter–Conway sequence and its centered transform, the Kolakoski sequence, and a cyclotomic-polynomial sequence. A sealed two-world continuation factorial over four state carriers and five update rules adds a complementary empirical result: 1,617 new A-numbers in union, with prospective predictive coding contributing the largest rule union (968) and exclusive marginal (299), but not winning every carrier–seed cell and remaining substantially more expensive than backpropagation. A separate two-round fresh-start comparison shows why target counts must be read through the full program–target relation: in round two Unified-PC covers 95 targets against Unified-BP's 82 while producing fewer fresh program identities (71 against 79), and one known program exclusively carries 36 of those targets. A one-program-per-target selected table preserves target coverage but omits 74 verified edges and 72 program identities in that round. A subsequent misspecified branch (footnoted in Section 11) added 747 further verified A-numbers to the pool; its update-rule comparisons are recorded but carry no evidential weight on the from-scratch question. A post-seal candidate-blind adjudication refuted 11 of 46 frozen candidate edges by independent executable replay; every refutation is an unbounded continuation of a finite entry, none a value disagreement inside the published range.

Open artifacts. The formal repository, paper source, compiled PDF, and neural experiment repository are public. The complete 33-program adjudication module and the reported-program correctness module, as well as the provenance-aware program-discovery development are linked separately. The theorem links below are immutable GitHub source permalinks to commit `f3a31db36bb2f944312ee31c1d86c2758109e83a`; they open the complete statement and proof, not a search page or a moving branch location.

1 Introduction

Begin with a single moment from the campaign this paper measures. A neural search, ranking one construction action at a time, assembles fourteen tokens into the program

$$\texttt{loop (mult (mult X X) X)} \circ (\texttt{addi} \circ \texttt{addi (addi X X) X})$$

and submits it against entry A016779 of the OEIS, whose definition reads $a(n) = (3n+1)^3$. The encyclopedia prints thirty-six terms; the program reproduces all thirty-six. What, exactly, do we know at that moment? We know the program agrees with a finite window — nothing more. Whether it computes $(3n+1)^3$ at $n = 100$, or at every n, is a different kind of question, and answering it needs a different kind of instrument: a semantics precise enough that "this program realizes this sequence on its whole domain" can be *proved*, and a graded notion of evidence for the vastly more numerous cases where proof has not yet been attempted. Keeping those two instruments separate — and honest about which one is speaking — is the discipline this paper builds. (For this particular program the strong instrument has since spoken: by Section 9 its claim is a kernel-checked theorem over the entire domain, not merely the printed window.)

The OEIS [4] is a natural benchmark for program synthesis: each entry is an integer sequence, and a solution is a program that reproduces its terms. Recent self-learning systems synthesize such programs at scale from a small combinator DSL, discovering programs for a large fraction of the OEIS without human-written training data [1, 2]. Our contribution is not another synthesizer but a *verified and measured* account of the objects a synthesizer manipulates.

We separate three concerns that are often conflated:

- **What a program *is*** — its syntax over a fixed operator vocabulary, and the machine-checked semantics of running it (Sections 2–3).
- **What a program *does on a probe*** — its input/output behaviour on a finite window (its *signature* σ), a finite observational summary (Section 5).
- **What properties static analysis can certify** — totality, sign, and parity: intensional information inferred conservatively rather than read off finitely many outputs (Section 6).

The payoff of keeping these apart is a clean division of labour: the probe layer gives cheap graded evidence but can never certify behaviour beyond the probe window; the intensional layer proposes conservative all-input certificates, while the Lean layer can turn selected claims into kernel proofs. Combined, they sharpen both benchmark hygiene and the discovery of conjectures.

2 The OEIS synthesis DSL

Programs are postfix token streams over a 16-operator vocabulary evaluated on a two-register integer state. The operators are constants `zero`, `one`, `two`; the index variable `x` $= n$ and an inner variable `y`; binary arithmetic `addi`, `diff`, `mult`, `divi`, `modu`; a conditional `cond` (test ≤ 0); bounded iterators `loop` and `loop2`; an unbounded search `compr` (the a-th $n \geq 0$ satisfying a predicate); and stack operators `push`, `pop`. Table 1 summarizes the arities and the partiality character of each operator, which is decisive for the analysis of Section 6.

A closed form illustrates the language: the factorial $n!$ is `loop(mult(x,y),x,one)` — fold multiplication by the running counter n times starting from 1. The counting sequence $a(n) = n+1$ is `addi(x,one)`.

operator	arity	role	partiality / termination
`zero`,`one`,`two`	0	constants	total
`x`,`y`	0	variables ($n \geq 0$; inner ≥ 0)	total
`addi`,`diff`,`mult`	2	arithmetic	total (resource: overflow)
`divi`,`modu`	2	division / modulo	*genuine*: error on zero divisor
`cond`	3	branch on ≤ 0	total given branches
`loop`,`loop2`	3, 5	bounded iteration	terminating (resource: timeout)
`compr`	2	unbounded search	*genuine*: may not terminate
`push`	2	stack push	total (resource: push limit)
`pop`	1	stack pop	*genuine*: error on empty

Table 1: The synthesis DSL. Only `divi`/`modu` (zero divisor), `pop` (empty stack), and `compr` (non-terminating search) introduce *genuine* partiality; `loop`/`loop2`, overflow, and timeouts are *resource* limits. This three-way character drives the totality analysis.

3 Verified GSLT semantics

The DSL is given a machine-checked operational semantics in Lean. On top of it we build a generative-semantics layer as a triple (T, E, R): terms T, an equational theory E (a setoid), and a rewrite relation R with left/right coherence laws. The coarse *big-step GSLT* rewrites a program to the integer value its fuel-bounded evaluator emits; a completed-value term is never reflexively a program, so non-triviality is load-bearing. Two modalities express synthesis goals:

$$\Diamond(\text{emits } v \text{ at } k) \iff \exists\, s',\ \texttt{eval}\ \textit{fuel}\ \textit{sig}\ p\ (\texttt{seed}\ k) = \texttt{some}(v, s'),$$
$$\texttt{EmitsPrefix}\ p\ \ell \iff \forall i\, v,\ \ell_i = v \Rightarrow \Diamond(\text{emits } v \text{ at } i).$$

`EmitsPrefix` is precisely the OEIS synthesis specification: "p reproduces the observed prefix ℓ."

Obligations. A synthesizer that claims correctness must respect a family of formal obligations. We single out three that are discharged as machine-checked theorems, axiom-clean (each theorem's `#print axioms` footprint lies within {propext, Classical.choice, Quot.sound}):

Obligation 1 (Recognizer completeness / mask equivalence)**.** *The legality predicate* ***`maskLegal`*** *on token streams coincides with the executable recognizer (**`recognize_iff_maskLegal`**), and every well-formed program is recognized (**`recognize_complete`**). Hence a constrained decoder that emits only mask-legal tokens can generate exactly the well-formed programs.*

Obligation 2 (Relabeling invariance)**.** *Evaluation is invariant under a signature isomorphism (**`eval_relabel_signature_iso`**): renaming primitives consistently does not change the computed sequence. Behaviour depends on structure, not on operator names.*

Obligation 3 (Canonicalization soundness)**.** *Flag-driven canonicalization preserves semantics; the relevant proofs are **`canonicalize_sound`** and **`orgCanonicalize_sound`**. Thus, for example, sorting the children of commutative operators cannot make canonical-form deduplication merge behaviourally distinct programs.*

Further obligations include equational-theory soundness and evaluation congruence for the modalities. The machine-checked `finite_probe_rigidity_negative_witness` already settles finite-probe rigidity in the negative: two programs can agree on a finite probe while differing

extensionally. This is the formal shadow of the leakage phenomenon in Section 5. A separate small-step account remains future work; the present rewrite layer deliberately uses the evaluator as its big-step relation.

4 The synthesizer as specification

The neural synthesizer maps a sequence prefix to a program graph (sequence-to-graph). Its decoder is *execution-guided*: at each step a legality mask restricts the token set to those that can extend a well-formed program, and the mask is exactly the recognizer of Obligation 1. Successive training variants add retrieval and contrastive objectives that consult a similarity index. The central design stance is that the network's operational contracts are the prover's theorems: the mask's soundness, the canonical form's soundness, and relabeling invariance are not engineering conveniences but the denotational safety envelope of the synthesizer.

5 Leakage-safe data and the probe-behaviour layer

Exact deduplication and splits. Each program's probe signature σ is its output vector on $n = 0, \ldots, 31$ at a fixed fuel, with $\perp$ (`None`) marking a non-value result at that budget. Programs with identical σ form an exact *probe-signature* class (called an "E-class" in the implementation); this is not extensional program equivalence. The train/val/test split is assigned by whole connected component so that no exact-signature duplicate straddles the split boundary. Over 104,083 programs this gate is leak-tight for the measured signatures: all 3,007 exact-σ duplicate pairs lie in the same split.

Graded similarity. Exact equality is brittle. We express graded similarity as a probabilistic-logic truth value $\langle s, c \rangle$ [7]: the probe-level strength is behavioural agreement over the *defined* domain, $s = |\text{agree}|/|\text{informative}|$, and confidence follows the chosen evidence-count law $c = m/(m + k)$, with m the number of informative probe positions and a fixed evidence scale $k > 0$. This is an evidence-weighting convention, not an empirical calibration claim. For fully defined signatures, exact equality gives $s = 1$, $m = 32$; when $\perp$ occurs, m counts only informative positions. A locality-sensitive index over signatures generates near-equivalent candidate pairs without an $O(n^2)$ scan.

Soft-leakage audit. The graded layer surfaces *near*-duplicates the exact gate cannot see. Table 2 reports pairs whose two programs sit in different splits. The benchmark-integrity figure is the fraction of evaluation programs with a near behavioural twin in *train*: at strength ≥ 0.95, 1.84% of test and 1.84% of val programs (lower bounds—the index can miss a pair, never invent one). A disagreement-locus analysis separates genuine near-twins (disagreements confined to a single term, none in the tail) from "false friends" that agree only on a shared trivial tail; the trustworthy signal concentrates in the ≥ 0.95 band. This vindicates gating on *exact* full-signature equality rather than a similarity threshold, which would either over-merge on trivial tails or miss single-term twins.

strength band	cross-split pairs	train↔test pairs
0.80–0.90	17,250	6,365
0.90–0.95	2,614	909
0.95–0.99	502	150

Table 2: Soft leakage: near-equivalent program pairs ($d \geq 1$, not byte-identical) placed in different splits by the exact gate. Distinct evaluation programs with a train near-twin: 1.84% (test and val) at ≥ 0.95; 5.4–5.8% at ≥ 0.90.

static property	count	share
total	37,996	36.5%
nonnegative (`nonn`)	45,251	43.5%
positive	14,907	14.3%
even	2,725	2.6%
odd	2,926	2.8%

Table 3: Static-property distribution over 104,083 programs. Shares are lower bounds under the analyzer's intended conservative interpretation.

6 The intensional layer: static properties and their proof boundary

Where the probe layer reads behaviour off finitely many outputs, the intensional layer uses abstract interpretation [5] to infer candidate all-input invariants. Its transfer functions are designed to over-approximate the concrete operators, and loops are handled by Kleene fixpoints in finite lattices. The current analyzer is implemented in Python and tested against the evaluator; its transfer functions have not yet been proved sound against the Lean semantics. We therefore distinguish its conservative static certificates from the kernel-checked program-correctness proofs of Section 9. We compute three properties.

Property 1 (Totality)**.** *A program is* classified total *when only resource limits can stop it: it contains neither* `compr` *nor* `pop`*, and every* `divi`*/*`modu` *divisor is certified nonzero. This is a conservative lower bound—"unknown" does not mean partial.*

Property 2 (Sign)**.** *A sign-domain analysis (with* $\mathtt{x} = n \geq 0$*, and* $a \bmod b \geq 0$ *for* $b > 0$*) certifies the OEIS keyword* `nonn` *(nonnegativity) for many programs under its abstract rules.*

Property 3 (Parity)**.** *A* $\mathbb{Z}/2$ *analysis certifies "always even" / "always odd" under its abstract rules (e.g.* `mult` *by an even factor is even).*

Validation gates. Two independent finite checks test the implementation; they do not replace a soundness proof. A static gate compares every sign/parity claim against the real signatures: 0 violations across all 104,083 programs ($\approx$ 3.3M value-checks). An empirical totality gate re-runs the real evaluator on 500 programs classified total to $n = 48$ (beyond the probe window): 0 genuine errors, only resource timeouts. Table 3 gives the distribution.

Disambiguating the signature. A $\perp$ in σ is ambiguous: is the program genuinely partial, or merely slow at the probe budget? Under the analyzer's intended soundness, a total classification

attributes it to resource exhaustion. Of the 12,966 programs carrying a $\perp$, 2,345 are classified total, while 10,621 are not, with attributed cause: 6,539 `compr` (non-terminating search), 3,023 `pop` (empty list), 1,059 `divi`/`modu` (possible zero divisor). The intensional layer thereby partitions a signal that finite execution alone leaves ambiguous.

7 Filtering and measuring in the OEIS loop

Both roles suggested by Table 3 are useful inside a synthesis loop.

Filtering. An OEIS target often carries stated properties—for example its keywords (`nonn`, `sign`, `mult`) or invariants justified by its defining formula. A candidate can be pruned soundly only when two premises are established: the target invariant is itself justified, and the analyzer proves an incompatible candidate invariant. An "unknown" abstract result must remain legal; requiring every candidate to be *proved* nonnegative would be incomplete and could discard a correct program. Thus, before the analyzer is connected to the decoder, uncertain properties may rank candidates, while hard pruning must be restricted to proved contradictions. A future Lean soundness theorem for the analyzer would make that restricted semantic mask recall-preserving on its stated domain.

Measuring. The same properties are theoretical measurements of a program and of a sequence. Totality analysis distinguishes a possibly partial sequence from one classified as slow-to-compute but total; the sign/parity results are static-analysis counterparts of OEIS keywords, so annotations can be predicted and checked at scale. More finely, the graded truth values give each program pair an evidence-weighted similarity, and formal proofs can supply factive intensional features. Revision is licensed only for source-disjoint packets: a proof must not be counted again as independent support for the translation or observations on which it depends. The corpus thereby becomes not merely programs and outputs, but programs, outputs, provenance, and proofs.

8 The loop compounds: a three-seed cumulative replication

The construction above is only interesting if the loop it serves actually runs. It does. We report a prospectively registered replication of the self-learning regime of [1] using a typed-action decoder over the verified skeleton of Section 4: the network never emits program text, only *legal construction actions* over an exact state, and a deterministic checker decides acceptance.

The protocol is cumulative. Each of three independently seeded worlds—with random seeds controlling initialisation, minibatch order, dropout, and search tie-breaking; the data pool (79,249 rows) and the 240 search tasks are shared—trains on its accumulated verified corpus, searches under a fixed budget, checks every candidate against the whole OEIS, adds newly covered sequences to its corpus, and repeats. The endpoint is cumulative coverage C_g and its increment $D_g = C_g - C_{g-1}$; no comparison against held-back external data governs continuation, since in the deployed regime no such reservoir exists.

The shared initial corpus is not chosen freely: it is the reference system's own public bootstrap seed `sol0` [1], 3,771 sequences found by *random* program search with no learning, and the very origin from which that system subsequently grew its full published corpus through many iterations of the loop we are running here. Starting from that seed rather than from a mature corpus is deliberate, with three consequences. The origin is reproducible and learning-free, a fixed public artefact anyone can replay from. Our run is then the *same* self-learning process re-executed—not a fork of another system's answers—so a rediscovery is a genuine held-out generalisation within our

own lineage rather than a lookup. And the base stays small enough that a world's own discoveries form a measurable share of the training signal instead of being drowned by it, which is what makes the loop's dynamics observable at all. Coverage per world:

world	initial	after gen. 1	after gen. 2
17	3,771	3,789	3,846
29	3,771	3,841	3,891
43	3,771	3,807	3,862

At generation three we registered, before any update, a three-way comparison of how the accumulated discoveries enter training: *uniform* replay (literal pooling, under which the 194 cumulative discoveries receive their natural 0.24% of sampling mass); *balanced* replay (25% discovery mass); and balanced replay with the auxiliary predictive heads disabled. All three arms continue from the same per-world parent checkpoint under identical update counts and search budgets, and are judged solely by new whole-OEIS A-numbers beyond the pooled 3,986 already known. New A-numbers, generation three:

setting	world 17	world 29	world 43	pooled
uniform replay	48	39	72	159
balanced replay, aux. off	65	155	88	308
balanced replay, aux. on	135	155	177	**467**

Every setting is positive in every world: nine of nine cells extend coverage, so the loop compounds rather than saturating at this scale. Two registered contrasts separate the mechanism. Balanced replay is worth +308 pooled over uniform—at literal pooling the model's own discoveries are numerically drowned by the base corpus, and the loop is starved of exactly the signal it exists to generate. The auxiliary heads are worth a further +159. The generation-three increment under the selected setting exceeds the generation-two increment by roughly threefold, and the setting carried into generation four was fixed by the registered selection rule rather than chosen after inspection.

Two negative results bound the claim, and we record them because they materially limit its interpretation. First, an ablation asking whether a world's own discoveries beat an equal mass of *unused external certified examples* answers no, in all three worlds (162 versus 253 pooled): feedback submitted more candidates yet explored fewer distinct programs (5,540 versus 6,233), a diversity contraction rather than a fitting failure. That comparison is a legitimate ablation and a poor objective—the fresh arm is an oracle-advantaged comparator that does not exist in the deployed loop, where the corpus is whatever the loop has verified. Second, no arm in any generation solved its own assigned held-out target within budget. The evidence therefore concerns learned *discovery* guidance over the catalogue—the regime [1] operates in—and not target-conditioned synthesis, which remains open.

What the verified layer contributes here is attributability rather than yield. Because the legality mask is machine-checked, a discovery cannot be an artefact of a malformed candidate slipping past a hand-written filter; because ranking is provably recall-preserving, guidance changes search order and cannot manufacture or destroy solutions; and because every accepted program is checked against the whole corpus, the increments above are exact catalogue-coverage counts under that checker rather than estimates or full-domain correctness claims. The loop's growth and the loop's trustworthiness are established by different machinery, and both are needed.

8.1 The compounding claim, tested against its own control

The replication above shares a weakness with the regime it replicates: cumulative coverage rises, but training time rises with it, so growth and continued optimisation are confounded. We therefore ran the compounding question separately, on a faithful port of the reference system's own architecture—a 10,781,697-parameter scaled-Luong bidirectional-LSTM sequence-to-sequence model—under the reference system's own protocol, in which every generation is trained *from fresh initialisation* and knowledge is carried by the accumulated corpus rather than by the weights.

That protocol makes an exact control available. In two independently initialised worlds, one arm trains generation two on the seed corpus *augmented* with generation one's own verified discoveries; the paired control trains on the seed corpus *alone* with fresh initialisation. Both are compute-matched, decoded and whole-OEIS-checked identically, and scored against the same fixed baseline of 5,631 known sequences, so the sole difference between them is whether the model's own discoveries entered its training data. That difference is the loop.

world	discoveries fed back	seed corpus only
1	1,980	375
2	2,051	519

Feeding verified discoveries back finds roughly four to five times the new frontier that re-seeding on the same corpus finds, in both worlds, with the pre-registered ordering met in each. Every count carries a content-addressed job identity and was reproduced by an independent verification pass. Because both arms start from fresh weights, no part of this gap can be attributed to accumulated optimisation: the corpus alone carries it.

The contrast with the replication above is itself informative, and we state it plainly rather than let it pass as an unnoticed deviation. The reference implementation re-initialises its network every generation; the typed-action campaign of this section continues weights and optimiser state across generations. The two designs answer different questions—one isolates the corpus, the other measures a lineage—and only the first can attribute compounding to feedback without qualification.

8.2 Eight generations, two worlds, four state carriers

The lineage design was then run to its registered endpoint: two independently initialised worlds, eight generations each, four matched-capacity decoder state carriers over the shared typed-action interface—a gated recurrent vector, a typed-slot workspace, and two belief-register variants (retention derived from the drift statistics versus learned). Every candidate was checked against the whole encyclopedia; all 64 cells passed independent verification. Distinct new sequences across everything: 984, from near-identical world totals (639 and 638, sharing only 293).

state carrier	distinct	found by no other	training cost
recurrent vector (GRU)	435	241	1.00×
typed workspace	383	171	2.85×
belief, derived retention	294	119	7.94×
belief, learned retention	244	138	3.53×

Three readings, in decreasing order of certainty. First, the yield ordering tracks the *decisiveness* ordering exactly—the recurrent vector had the lowest action entropy, the highest gold-action probability, and the fewest duplicate proposals—which is the mechanism the `equal_crossEntropy_`

strictly_ordered_expectedYield theorem isolates under a fixed search budget. The broader fixed-budget scope and its counterexamples are collected by budgetedSearchYield_crown. Second, 669 of 984 sequences were found by exactly one state carrier, with pairwise overlaps of 8–16%: the carriers steer search into substantially different territory, so the union is worth far more than the best single arm, yet a fixed-split portfolio under one budget reached only 399 against the best arm's 435—complementarity is real and naive allocation fails to harvest it. Third, and most tentatively, the derived-retention belief arm (retaining $\approx 94\%$ of prior evidence per revision) out-yielded the learned-retention arm, which learned to discard roughly half its evidence per revision—the direction the drift theory predicts in a near-stationary domain, where the optimal retention approaches one. Margins varied across the two worlds (one a clear recurrent-vector win, the other a near-tie), so these are descriptive statements about two runs, not population claims. Roughly three quarters of every arm's search budget was spent re-proposing programs it had already generated, which bounds how much any architectural conclusion can matter before search-side deduplication is addressed.

8.3 Two more generations: carrier by credit rule

The next registered campaign keeps the four generation-eight parents frozen and trains only a shared-form zero-output residual adapter. It crosses the four carriers with five update rules—ordinary BP, an independent BP control, prospective PC, deep error-coordinate PC and incremental PC—in two worlds. Every rule begins from the same parent-specific update-zero state and data ledger. Generation ten continues the exact generation-nine adapter and optimizer, and all accepted programs again pass the whole-OEIS checker. Thus the experiment measures a cumulative adapter lineage rather than replacing the original carrier campaign or the fresh-initialization compounding control.

Across the sealed generation-nine/ten surface, the cross-world union contains 1,617 new A-numbers. The update-rule portfolios are:

update rule	distinct new entries	exclusive marginal
prospective PC	968	299
error-coordinate PC	893	66
ordinary BP	869	51
independent BP	866	77
incremental PC	693	124

The same cells grouped by carrier give 738/396 for the recurrent vector, 643/281 for derived belief, 564/220 for learned belief and 517/183 for the typed workspace, where each pair is union/exclusive marginal. These are not additive counts: their overlaps are precisely why the full union is the primary endpoint.

Prospective PC supplies the largest rule union and exclusive marginal, but the matched comparisons expose a carrier–world interaction rather than universal dominance. It leads ordinary BP on three carriers and ties one in world two; in world one it leads only learned belief, ties derived belief and trails on the other two. BP also remains substantially faster per training hour. The robust empirical statement is therefore that the rules and carriers explore complementary verified-program regions under continued adapter learning.

The terminal result is the checker-audited evidence chain, produced after seven single-owner replays isolated a shared-channel overwrite hazard. Its independent verifier passes 749/749 checks and has SHA-256 `3f08e6c3a57ba3e13a0e387737bbb31cfd990221a7b3b331214b4f9cab47bc7f`. The numerical verdict it authenticates has SHA-256 `02ca3cb75003f18678c5fcd43d93c07da584a365ffec709510a5c7147f8491a6`. The

ordinary and audited numerical summaries agree; the isolated chain is authoritative because provenance, not numerical coincidence, determines which checker evidence may support the claim.

The next allocation decision is now stated as a theorem rather than an informal ranking. For target value v, arm costs c_a, and a selected portfolio P, define

$$U(P) = v \left| \bigcup_{a \in P} A_a \right| - \sum_{a \in P} c_a,$$

where A_a is the arm's checker-accepted set. Adding a fresh arm changes utility by exactly v times its marginal coverage minus its cost (`portfolioNetValue_insert_eq_add_marginalNetValue`). Consequently a candidate is preferred to another independent BP seed exactly when its marginal net value is larger (`insert_candidate_gt_insert_opportunity_iff`). The executable positive fixture shows a lower-standalone-yield PC arm winning through exclusive targets; the negative fixture shows the same exclusivity losing once its cost is too high. This licenses complementary arms without treating complementarity as a blank cheque.

8.4 A fresh four-arm round, and what its headline count conceals

The preceding campaigns all continued an existing parameter lineage. The next one does not: every arm begins from freshly initialized weights, so that the update rule rather than an inherited state is the object under comparison. Four arms are trained concurrently—two carriers (a unified multi-site carrier and a coarser recurrent sequence-to-sequence carrier) crossed with ordinary BP and prospective PC—from one shared initialization per carrier, with a common pool, checker and union step. The round contributes pool event fourteen; we retain both numberings because the pool accounting and the training rounds no longer advance together.

The cross-arm union contains 493 new A-numbers. The per-arm counts are 336 for recurrent BP, 34 for recurrent PC, 70 for unified BP and 73 for unified PC. Taken alone, that table says the recurrent BP arm dominates by an order of magnitude. Three further measurements say it does not mean what it appears to.

Almost all of the headline is repertoire reproduction. Restricting to sequences that actually appeared in optimizer batches and asking whether the accepted candidate reproduced a program already in the catalog gives 210/265 (79.2%) for recurrent BP, 2/17 (11.8%) for recurrent PC, 21/44 (47.7%) for unified BP and 23/48 (47.9%) for unified PC. The largest count is therefore also the most reproductive: it is exceptional at recovering the learned program repertoire, which is a real capability but a different one from discovery.

Strict novelty inverts part of the ranking. Of the 493, only 35 are sequences absent from the entire round-one catalog: 18 from recurrent BP, 7 from recurrent PC, 6 from unified BP and 4 from unified PC. As a fraction of each arm's own yield that is 5.4%, 20.6%, 8.6% and 5.5%: the arm with the smallest absolute count has by far the largest novelty share. Every one of the 35 was exclusive to a single arm.

The two recurrent arms share nothing. Their solved sets intersect in zero A-numbers—33 of recurrent PC's 34 are exclusive across all four arms—so the low-yield arm is complementary rather than a degraded copy of its partner. This is precisely the configuration the portfolio theorem above was written for: marginal coverage, not standalone yield, is what a further arm must be judged on.

Discovery is collateral, not conditional. The sharpest result concerns none of the arms individually. Of 240 queried target sequences, direct hits number 6, 0, 8 and 8 respectively—at most 3.3%. Not one of the 493 accepted discoveries was for the sequence being requested when it was generated. Every one was found by the checker noticing that a program generated for some other target also explains it. Meanwhile the same runs cover 1,206, 355, 1,619 and 1,780 catalog sequences in total.

The synthesizers therefore behave as strong *unconditional* program priors with weak conditioning on the request. Asked for a program for a particular sequence they mostly fail; allowed to generate and let the checker attribute, they are productive. That separates two estimands which the literature routinely reports as one, and we report them separately hereafter: *local specificity*, the rate at which a requested target is solved, and *global utility*, the verified coverage a run produces regardless of what was asked. Improving the first is a conditioning problem and is not addressed by anything that improves the second.

The matched pair. Within the unified carrier, where search work is matched to within 0.1% (129.52 against 129.64 GPU-seconds), PC leads BP on global coverage (1,780 against 1,619), on additions to the pool (73 against 70), on unique candidate programs (16,265 against 14,431) and on accepted programs with coverage (7,447 against 6,315), while tying on direct target hits (8 each) and trailing on strictly catalog-new sequences (4 against 6). Its training cost was 3.83 times higher: 36.25 hours against 9.46. This is a single seed and a descriptive result, not an efficacy claim; we record it because the matched-work condition makes it interpretable and the cost ratio makes it consequential.

8.5 Round two: facts, programs, targets, and one essential hub

The same four lineages continued for a second fixed round. The external checker again searched every proposed program against every eligible OEIS target. Per-arm new-target coverage was recurrent BP 215, recurrent PC 17, Unified-BP 82, and Unified-PC 95; directly requested hits were 4, 0, 5, and 8 of 240. The recurrent PC regression is therefore broad, whereas the Unified PC arm leads its matched BP arm on both registered target endpoints.

Target count is only one projection of what the checker certifies. Let $R \subseteq \mathsf{Program} \times \mathsf{Target}$ be the complete finite relation of verified new facts. Then $|R|$, $|\pi_{\mathsf{P}} R|$, and $|\pi_{\mathsf{T}} R|$ count facts, program identities, and target identities, respectively. They are not interchangeable Bernoulli atoms. For any projection π, the exact finite law is

$$|\pi[A \cup B]| + |\pi[A] \cap \pi[B]| = |\pi[A]| + |\pi[B]|,$$

and projected evidence is additive exactly when the projected footprints are disjoint (`projected_inclusion_exclusion`, `projected_union_eq_sum_iff`). Distinct source records are not enough: they may collide after projection to one program, target, or provenance root.

Reconstructing every checker witness gives:

round	arm	facts	programs	targets	fresh programs
1	recurrent BP	393	312	336	110
1	recurrent PC	38	35	34	32
1	Unified-BP	96	95	70	69
1	Unified-PC	104	101	73	74
2	recurrent BP	224	211	215	93
2	recurrent PC	18	17	17	15
2	Unified-BP	115	113	82	79
2	Unified-PC	125	89	95	71

The one-representative-per-target union is a view, not this ledger. In round one it retains 493 of 612 facts and 445 of 525 programs while preserving all 493 targets. In round two it retains 396 of 470 facts and 346 of 418 programs while preserving all 396 targets. The exact completeness criterion is program-set equality, not target completeness (`curated_programCoverage_preserved_iff`).

The complete round-two Unified relation localizes the apparent PC gain. Unified-BP's 82 targets are supported by 34 same-target catalog witnesses, no cross-target reuse, and 54 fresh-program witnesses, with six targets in both the first and third classes. Unified-PC's 95 consist of 17 same-target catalog, 35 cross-target reuse, and 49 fresh-program-supported targets, again with six in the overlap. Thus the +13 target contrast decomposes as $-17 + 35 - 5$ with equal overlap correction. Unified-PC does not generate more fresh program identities; it reuses known structure across more targets.

That reuse is concentrated. One known Unified-PC program has target degree 36—one same-target catalog fact and 35 cross-target facts—and every one of those targets is exclusive to it. Removing the identity changes Unified-PC coverage from 95 to 59; removing the most essential Unified-BP identity changes 82 to 80. Formally, if $e_R(p)$ is the number of targets lost on removing program p, then

$$|\pi_{\mathsf{T}}(R \setminus p)| + e_R(p) = |\pi_{\mathsf{T}} R|, \qquad e_R(p) \le |\{t : (p,t) \in R\}|,$$

with both identities machine checked (`targetCoverage_withoutProgram_add_exclusive`, `exclusiveTargetContribution_le_programTargetDegree`). The inequality can be strict: round-one recurrent BP contains two degree-35 programs with zero exclusive contribution because their targets have redundant witnesses.

The program-trial rate agrees with the concentration diagnosis. Round-two new-target covering programs are 113/17133 = 0.006595 for Unified-BP and 89/15136 = 0.005880 for Unified-PC. The registered target gain is real, but it is a high-multiplicity reuse event rather than a broad increase in program-level hit rate. The witness-complete summary has SHA-256 `44ac419d92ab625bce23b0356ba52b7bff8c7e516f93cba370495906ea7cee63`; the architecture sensitivity summary has SHA-256 `9dd87c6c740f0301b56260c13882332082a556c4d548f12936777ce70c23619e`.

8.6 An equal-work joint-retraining control

Four rounds compared credit rules under continuation: every arm inherited its own weights across rounds. Nothing in that design tests continuation itself. Two further lineages were therefore registered as observers, one per BP carrier, each trained from the authenticated initial weights on the frozen final cumulative population for 17,500 updates—the cumulative budget the continued lineages had spent by round four. Observer proposals were searched and checked exactly as the registered arms are, and their discoveries were quarantined from the shared event sequence, so the control cannot alter the lineage it measures.

The estimand needs stating precisely, because two things change together. The observer sees the final population from its first update, whereas the continued lineage saw each round's evidence only once that round produced it. The comparison is therefore between offline joint retraining on the accumulated corpus and streaming continuation over the same corpus, not between fresh and inherited weights with the data schedule held fixed. Snapshots at 10,000, 12,500, and 15,000 updates match the cumulative budgets of rounds one through three, but they are future-informed budget landmarks rather than replicas of those rounds; only the terminal comparison is free of that advantage.

Writing J for joint retraining and C for continuation, with every entry recomputed from the checker's own relation:

round	carrier	new J/C	direct J/C	existing J/C	candidates J/C	yield ratio	$\lvert J \cap C \rvert$
1	recurrent	361/336	4/6	629/870	2376/2564	1.16	109
1	Unified	77/70	4/8	1603/1549	14758/14431	1.08	9
2	recurrent	304/215	6/4	842/907	2582/2645	1.45	50
2	Unified	77/82	7/5	1388/1758	12967/17133	1.24	13
3	recurrent	228/192	7/7	882/1047	2595/2752	1.26	39
3	Unified	52/44	11/10	1689/1732	15710/16504	1.24	7
4	recurrent	225/202	8/6	959/1079	2814/2784	1.10	17
4	Unified	64/76	10/10	1567/1837	15244/19701	1.09	5

The round-one evaluation predates the direct-hit field, so both round-one C entries were recomputed from the generation and coverage relations under the same definition used elsewhere: a target counts as direct when a program proposed *for* that target solves it.

Three readings are stable across the eight cells, and one is not.

New-target yield per unique proposed program favours J in every cell, by a factor between 1.08 and 1.45. Joint retraining is the more efficient proposer per candidate emitted. Retention runs the other way: continuation covers more already-known targets in seven of eight cells, by 370 and 270 targets in the two widest Unified cells, and total support—new plus existing—favours continuation in six of eight. Efficiency and breadth are separated, and they point in opposite directions.

Absolute new-target count is architecture-dependent, and this is where care is needed. For the recurrent carrier J leads in every round, by $+25$, $+89$, $+36$, and $+23$. For the Unified carrier the sign alternates: $+7$, -5, $+8$, -12. Only round four is free of the observer's future-information advantage, and there the carriers disagree, $+23$ against -12, a difference in differences of $+35$. Because the landmark advantage favours J, the round-four Unified deficit is conservative, while the recurrent result is the weaker of the two, its sign being the one that bias would produce. Four checkpoints of one pair of training runs are not four replications, and nothing here is a population claim.

Hub robustness separates the round-four results further. With $e_R(p)$ the exclusive contribution of program p as above, the worst single-program removal costs joint-retrained Unified 7 of its 64 targets and continued Unified 2 of its 76; both recurrent cells lose 2. Continued Unified's larger count is also the more distributed one, and the comparison would invert under a hub-discounted reading only for the smaller arm.

Witness redundancy explains why removal costs so little on the recurrent side. Counting new targets supported by two or more distinct program identities, the round-four Unified cells carry 19 of 64 and 25 of 76—about a third—while the recurrent cells carry 14 of 225 and 10 of 202, near a twentieth. The recurrent carrier finds many targets each by a single route; the Unified carrier finds

fewer, more often by several. Removing one identity is cheap in the first case because the targets are nearly disjoint across programs, and cheap in the second because the survivors re-cover what was removed—the same robustness arising from opposite relational structures, which is why the bare deletion number needs the redundancy distribution beside it to be read at all.

The portfolio structure is the most robust feature of the table. The two conditions agree on very little—the Jaccard index of their new-target sets runs from 0.185 down to 0.037, falling monotonically for the recurrent carrier—so the union exceeds the better single condition by a factor between 1.54 and 1.82 in all eight cells: at round four, 410 recurrent targets against the best single 225, and 135 Unified against 76. Treating the conditions as capture and recapture on the reachable undiscovered pool gives Lincoln–Petersen estimates of 2674 and 973 targets at round four. Shared architecture and shared corpus make the two draws positively dependent, so these are lower bounds on the reachable pool rather than estimates of it, and the projection caveats of `projected_inclusion_exclusion` apply to the union counts exactly as before.

The machine-readable comparison has SHA-256 `78ba544621388a32fa1b0d3e71fb650b55cdb75224f71430ceb27da46011f1f9`; the independent recount from raw checker rows has SHA-256 `9a85475b7de4301f324121d89f4b9ba6a168c452c3a27c9f307a595dc733c`

9 From coverage to proof: adjudicating discoveries against their definitions

Everything above establishes that a discovered program reproduces a sequence's stored terms. That is weaker than it sounds. The published entries are short—median 38 terms corpus-wide, and only 26 for the sequences our own loop discovered, with 22% resting on fewer than twenty terms and one on three. This gap is consistent with a simple selection mechanism: short entries impose fewer observed constraints and are therefore more vulnerable to coincidental matches. And finite agreement is exactly what our finite-probe rigidity counterexample proves insufficient—agreement on a finite prefix need not force extensional equality.

So we asked the question the loop cannot answer by itself: does the discovered program actually compute the sequence its entry *defines*? This is answerable, because the encyclopedia carries far more than terms. Of our 984 discoveries, all 984 have keywords, 88% an explicit formula field, 88% a reference program, and 75% prose commentary. These fields provide candidate sources for specifications, but their precision varies: a closed formula is much less ambiguous than a keyword or informal comment.

Protocol. A definition can always be made to match a program after the fact, so the order of work is the whole methodology. Candidate programs were frozen first, from the completed campaign, as exact generated token streams with their hashes. Sequences were then selected on *metadata alone*, and each specification was formalized and locked *before* its candidate program was revealed. The specification therefore states a proof obligation rather than describing a known solution. This is the ordinary discipline of verification—a specification confronting a pre-existing implementation—and not, as one might worry, a translation from definition to program.

What is proved. For a specification with domain D, index map, and value function, and a frozen token stream t:

$$\text{Realizes}(\text{spec}, p) \;\equiv\; \forall\, i,\; D(\text{index}(i)) \;\to\; \text{EventuallyEmits}\big(p,\, i,\, \text{value}(\text{index}(i))\big),$$

where EventuallyEmits requires a fuel threshold beyond which the interpreter *stably* returns that value. Three things deserve emphasis. The quantifier is over the entire declared domain, not a

probed window, so this is extensional correctness rather than extended testing. Termination is part of the claim: fuel stability is proved, not assumed, and a companion theorem establishes that a successful evaluation is unique across sufficient fuel (`eventuallyEmits_unique`). The formal definitions are `EventuallyEmits`, `Realizes`, and `CandidateRealizes`. Finally, the verified program is not a manual Lean reimplementation: the frozen token stream is parsed by the same recognizer the synthesizer uses, with the parse discharged by kernel computation, so what is verified is the artifact the network emitted.

Result. Thirty-three sequences were adjudicated, and every frozen program associated with them was proved extensionally correct on its full domain. Each carries an axiom audit; all depend only on the standard logical basis. Each record pins the OEIS identifier, entry revision, entry digest, offset, token stream and program hash, so a reader can reconstruct exactly which published text was translated. The registry cardinality is itself checked by `certifiedProofPacket_count`, and all 33 individual proofs are in the linked adjudication module above.

The OEIS definitions and metadata quoted below are attributed to the OEIS Foundation Inc. [4], from the pinned `oeisdata` snapshot at revision `a6e0f22854cc1c307da428e9d6295093781df7fa`, and are used under the Creative Commons Attribution-ShareAlike 4.0 license. Each formalization additionally stores the exact entry digest rather than relying on the repository revision alone.

The five examples below are deliberately not compressed into one table. For readability, write $\mathrm{Loop}(f, k, z) = f^{\circ k}(z)$ for these loop bodies, all of which ignore the loop counter, and abbreviate the Lean namespace `GauthierE2.P` as `P`. The displayed "semantic normal form" is the value established by the proof, not merely a pretty-printing or a finite simplifier run. Each separate Lean listing then shows the source specification, frozen hash and tokens, parsed program AST, recognizer equality, literal theorem type, and the theorem with all three semantic definitions unfolded. All five specifications have offset zero, so the visible domain premise $0 \leq \mathrm{Int.ofNat}(\mathrm{position})$ is automatic; we retain it to expose the domain-aware theorem exactly.

A016779: one cubing step. The discovered GSLT program has the readable semantic normal form

$$p_{28}(n) = \mathrm{Loop}(x \mapsto x^3, 1, 1 + 3n) = (3n + 1)^3.$$

Full machine-checked statement and proof: `A016779_candidate28_correct`.

Listing 1: A016779, discovered exclusively by the recurrent-vector arm in world 1, generation 8. The proof ranges over the entire domain, not only the 36 published terms.

```
-- OEIS %N: a(n) = (3*n + 1)^3. %O 0,2
def A016779.source := sourceOf "A016779"
  "56bc1bf16ed408a98b0d997bc8006d1b34e8796dcaf47bfbd91586155f526b97" 0
def A016779.spec := specOf 0 (fun n => (3 * n + 1) ^ 3)

def candidate28 : FrozenCandidate where
  programSha256 := "dba931cee529c62f5c8284b78fbb0974327221981a029c00f99a6b4d0711eb2d"
  tokens := [10, 10, 5, 10, 5, 1, 1, 10, 10, 3, 10, 3, 3, 9]
  program := P.loop (P.mult (P.mult P.X P.X) P.X) P.o
    (P.addi P.o (P.addi (P.addi P.X P.X) P.X))
  recognized := rfl

#check A016779_candidate28_correct :
  CandidateRealizes A016779.spec candidate28

-- The same theorem, fully unfolded and kernel-checked:
example : forall position : Nat, (0 : Int) <= Int.ofNat position ->
    exists threshold, forall fuel, threshold <= fuel ->
      GauthierE2.term fuel orgMemoSignature candidate28.program
        (Int.ofNat position) =
          some ((3 * Int.ofNat position + 1) ^ 3) := by
  simpa [CandidateRealizes, Realizes, EventuallyEmits,
    A016779.spec, specOf, SequenceSpec.index]
    using A016779_candidate28_correct
```

A016780: two squaring steps. Here the network expresses a fourth power as two iterations of squaring:

$$p_{29}(n) = \mathrm{Loop}(x \mapsto x^2, 2, 1 + 3n) = \left((3n+1)^2\right)^2 = (3n+1)^4.$$

Full machine-checked statement and proof: A016780_candidate29_correct.

Listing 2: A016780, discovered exclusively by the belief-register arm through derived retention in world 1, generation 4. The OEIS entry contains 31 published terms; the theorem is universal.

```
-- OEIS %N: a(n) = (3*n + 1)^4. %O 0,2
def A016780.source := sourceOf "A016780"
  "86c883b0debc37c8ada7ab8f7e0414d0f6f156b2533f6090d993718f45737168" 0
def A016780.spec := specOf 0 (fun n => (3 * n + 1) ^ 4)

def candidate29 : FrozenCandidate where
  programSha256 := "c57882d00078998c452c5feee21dc0c7d0399e0f60379d118fd421c54acf6d67"
  tokens := [10, 10, 5, 2, 1, 1, 2, 3, 10, 5, 3, 9]
  program := P.loop (P.mult P.X P.X) P.tw
    (P.addi P.o (P.mult (P.addi P.o P.tw) P.X))
  recognized := rfl

#check A016780_candidate29_correct :
  CandidateRealizes A016780.spec candidate29

-- The same theorem, fully unfolded and kernel-checked:
example : forall position : Nat, (0 : Int) <= Int.ofNat position ->
    exists threshold, forall fuel, threshold <= fuel ->
      GauthierE2.term fuel orgMemoSignature candidate29.program
        (Int.ofNat position) =
          some ((3 * Int.ofNat position + 1) ^ 4) := by
  simpa [CandidateRealizes, Realizes, EventuallyEmits,
    A016780.spec, specOf, SequenceSpec.index]
    using A016780_candidate29_correct
```

A016814: one squaring step. The initial expression changes, while the learned loop skeleton is reused:

$$p_{30}(n) = \mathrm{Loop}(x \mapsto x^2, 1, 1 + 4n) = (4n + 1)^2.$$

Full machine-checked statement and proof: A016814_candidate30_correct.

Listing 3: A016814, discovered exclusively by the belief-register arm through derived retention in world 2, generation 1. The proof replaces a 44-term match with a full-domain theorem.

```
-- OEIS %N: a(n) = (4*n + 1)^2. %O 0,2
def A016814.source := sourceOf "A016814"
  "2bb3a661541a70b58c6a0e2fd6b96931502217d3eead53dd16a0d04607b4a3a2" 0
def A016814.spec := specOf 0 (fun n => (4 * n + 1) ^ 2)

def candidate30 : FrozenCandidate where
  programSha256 := "49e6cea5ab07ccf1dfac616e36e42e5e49c7ed074c227c30ddf946f24587efef"
  tokens := [10, 10, 5, 1, 1, 2, 2, 3, 10, 5, 3, 9]
  program := P.loop (P.mult P.X P.X) P.o
    (P.addi P.o (P.mult (P.addi P.tw P.tw) P.X))
  recognized := rfl

#check A016814_candidate30_correct :
  CandidateRealizes A016814.spec candidate30

-- The same theorem, fully unfolded and kernel-checked:
example : forall position : Nat, (0 : Int) <= Int.ofNat position ->
    exists threshold, forall fuel, threshold <= fuel ->
      GauthierE2.term fuel orgMemoSignature candidate30.program
        (Int.ofNat position) =
          some ((4 * Int.ofNat position + 1) ^ 2) := by
  simpa [CandidateRealizes, Realizes, EventuallyEmits,
    A016814.spec, specOf, SequenceSpec.index]
    using A016814_candidate30_correct
```

A070439: a computed modulus. The program does not contain the literal 16; it constructs it by squaring twice from two:

$$p_{45}(n) = n^2 \bmod \mathrm{Loop}(x \mapsto x^2, 2, 2) = n^2 \bmod 16.$$

Full machine-checked statement and proof: `A070439_candidate45_correct`.

Listing 4: A070439, discovered exclusively by the recurrent-vector arm in world 1, generation 5. The synthesized denominator computes 16 rather than storing it.

```
-- OEIS %N: a(n) = n^2 mod 16. %O 0,3 (101 published terms)
def A070439.source := sourceOf "A070439"
  "df5acb01e6c2570f6b4592782d285292d0cb69a4cd1987052c2f5ac6b0cc6de4" 0
def A070439.spec := specOf 0 (fun n => n ^ 2 % 16)

def candidate45 : FrozenCandidate where
  programSha256 := "315061126a0a0fb9eb5c6b0de9be9daa0977d662244d07020ff5276021a5f81e"
  tokens := [10, 10, 5, 10, 10, 5, 2, 2, 9, 7]
  program := P.modu (P.mult P.X P.X)
    (P.loop (P.mult P.X P.X) P.tw P.tw)
  recognized := rfl

#check A070439_candidate45_correct :
  CandidateRealizes A070439.spec candidate45

-- The same theorem, fully unfolded and kernel-checked:
example : forall position : Nat, (0 : Int) <= Int.ofNat position ->
    exists threshold, forall fuel, threshold <= fuel ->
      GauthierE2.term fuel orgMemoSignature candidate45.program
        (Int.ofNat position) =
          some (Int.ofNat position ^ 2 % 16) := by
  simpa [CandidateRealizes, Realizes, EventuallyEmits,
    A070439.spec, specOf, SequenceSpec.index]
    using A070439_candidate45_correct
```

A070478: the cubed companion. The same learned construction of 16 is paired with a cubed numerator:

$$p_{46}(n) = n^3 \bmod \mathrm{Loop}(x \mapsto x^2, 2, 2) = n^3 \bmod 16.$$

Full machine-checked statement and proof: `A070478_candidate46_correct`.

Listing 5: A070478, discovered exclusively by the recurrent-vector arm in world 1, generation 2. Its 96 published terms become a universal stable-evaluation claim.

```
-- OEIS %N: a(n) = n^3 mod 16. %O 0,3
def A070478.source := sourceOf "A070478"
  "cd9f8752f701420ceb43a72e77db95d1d3816fddc6db55c8b531aabbab41947b" 0
def A070478.spec := specOf 0 (fun n => n ^ 3 % 16)

def candidate46 : FrozenCandidate where
  programSha256 := "310209b1db332a56d73abbd2476b62292b37548b20f7114124632ba8cdbb61cc"
  tokens := [10, 10, 5, 10, 5, 10, 10, 5, 2, 2, 9, 7]
  program := P.modu (P.mult (P.mult P.X P.X) P.X)
    (P.loop (P.mult P.X P.X) P.tw P.tw)
  recognized := rfl

#check A070478_candidate46_correct :
  CandidateRealizes A070478.spec candidate46

-- The same theorem, fully unfolded and kernel-checked:
example : forall position : Nat, (0 : Int) <= Int.ofNat position ->
    exists threshold, forall fuel, threshold <= fuel ->
      GauthierE2.term fuel orgMemoSignature candidate46.program
        (Int.ofNat position) =
          some (Int.ofNat position ^ 3 % 16) := by
  simpa [CandidateRealizes, Realizes, EventuallyEmits,
    A070478.spec, specOf, SequenceSpec.index]
    using A070478_candidate46_correct
```

Two details in these listings repay attention. The programs need not mirror the definitions syntactically: A070439's specification is $n^2 \bmod 16$, while the discovered program computes the modulus as `loop (mult X X) tw tw`—sixteen obtained by iterated squaring from two, rather than written down—and the proof must reconcile the two. And the discovering arms are not the ones the headline count favours: two of these five came from the belief-register architecture, which finished *last* on raw yield (294 against the recurrent vector's 435). All five were found by exactly one architecture. The diversity argument of §8.2 was until now a statement about counts; here it is a statement about programs that are provably correct.

9.1 Independent verification of four previously reported programs

Urban's SCML'26 presentation, joint work with Gauthier, Olšák, and Hales, collects unusual programs produced by the Alien Coding line of work and asks how such programs can be explained and proved correct [1, 3]. We independently formalized four of the displayed programs using the exact E2 operator semantics introduced above. These programs are historical external examples, not outputs of our neural campaign: they are excluded from the 984 discoveries, all training pools, and every architecture comparison. The attribution is therefore both substantive and methodologically clean.

The proof target is again `CandidateRealizes`, not agreement on a longer prefix. Each theorem quantifies over every position in the sequence domain and establishes stable evaluation beyond a fuel threshold. The stored SHA-256 values authenticate our exact program transcriptions; because the original run artifacts were not available, we do not claim that these hashes authenticate the authors' historical files. Each example below has its own listing so that the readable program, frozen token stream, Lean AST, and complete top-level theorem statement remain visible together.

A004001: the Hofstadter–Conway recurrence. The reported program is

```
loop(push(loop(pop(x), y-x, pop(x)), x)
    + loop(pop(x), x-1, x), x-1, 1)
```

and the verified specification is $a(1) = a(2) = 1$ and $a(n) = a(a(n-1)) + a(n-a(n-1))$ for $n \geq 3$.

Listing 6: The reported A004001 artifact and its universal Lean theorem.

```
def reportedA004001 : FrozenCandidate where
  programSha256 :=
    "7520be9f41f5ef978fb00080a0130143616cadef82c39163b08276a081e5f23b"
  tokens := [10,15,11,10,4,10,15,9,10,14,10,15,10,1,4,
    10,9,3,10,1,4,1,9]
  program := P.loop
    (P.addi
      (P.push (P.loop (P.pop P.X) (P.diff P.Y P.X) (P.pop P.X)) P.X)
      (P.loop (P.pop P.X) (P.diff P.X P.o) P.X))
    (P.diff P.X P.o) P.o
  recognized := rfl

theorem reportedA004001_correct :
    CandidateRealizes specA004001 reportedA004001
```

Full machine-checked statement and proof: `reportedA004001_correct`.

A004074: the Hofstadter–Conway $10,000 sequence. This program computes the centered transform $2A004001(n) - n$:

```
((2 * loop(push(loop(pop(x), x-1, x), x)
         + loop(pop(x), y-x, pop(x)), x-1, 1)) - 1) - x
```

The theorem connects that operational program to the recurrence-defined A004001 formalization, so the transform is not justified by treating the displayed terms as a lookup table.

Listing 7: The reported A004074 artifact and its universal Lean theorem.

```
def reportedA004074 : FrozenCandidate where
  programSha256 :=
    "5d0a2d05c698935364899e73b16259ecc7c12a6319bd5a0fbc497ed6f3b90c51"
  tokens := [2,10,15,10,1,4,10,9,10,14,10,15,11,10,4,
    10,15,9,3,10,1,4,1,9,5,1,4,10,4]
  program := P.diff
    (P.diff
      (P.mult P.tw
        (P.loop
          (P.addi
            (P.push (P.loop (P.pop P.X) (P.diff P.X P.o) P.X) P.X)
            (P.loop (P.pop P.X) (P.diff P.Y P.X) (P.pop P.X)))
          (P.diff P.X P.o) P.o))
      P.o)
    P.X
  recognized := rfl

theorem reportedA004074_correct :
    CandidateRealizes specA004074 reportedA004074
```

Full machine-checked statement and proof: `reportedA004074_correct`.

A000002: Kolakoski's self-describing run lengths. The reported program is

```
1 + pop(loop(push(loop(pop(x), x div 2, x),
                 (x + pop(x)) mod 2) + x,
            x, push(1, 0)))
```

The specification is intensional: the values lie in $\{1, 2\}$ and their successive run lengths reproduce the sequence itself. Thus the proof must expose the program's packed cursor and show that its state transition maintains the run boundary, not merely identify the first several hundred outputs.

Listing 8: The reported Kolakoski artifact and its universal Lean theorem.

```
def reportedA000002 : FrozenCandidate where
  programSha256 :=
    "b6765feccadb491981a6a8a087ac78ebe98071e6b25f002f10364a7576b6ac6c"
  tokens := [1,10,15,10,2,6,10,9,10,10,15,3,2,7,14,10,
    3,10,1,0,14,9,15,3]
  program := P.addi P.o
    (P.pop
      (P.loop
        (P.addi
          (P.push
            (P.loop (P.pop P.X) (P.divi P.X P.tw) P.X)
            (P.modu (P.addi P.X (P.pop P.X)) P.tw))
          P.X)
        P.X
        (P.push P.o P.z)))
  recognized := rfl

theorem reportedA000002_correct :
    CandidateRealizes Kolakoski.specA000002 reportedA000002
```

Full machine-checked statement and proof: `reportedA000002_correct`.

A070526: cyclotomic values. For $n \geq 1$, the OEIS specification is $a(n) = \Phi_n(2^n)$. The reported program is

```
loop(loop2(if (x mod y) <= 0 then (x div y) else x,
          pop(y), y,
          push(loop(1 + (x + x), y, 1), x), x),
     (2 + x) * x, 1)
```

Its outer loop constructs Mersenne numbers and repeatedly removes earlier factors. The proof requires an order-sensitive all-history sieve theorem: the tempting independent claim $k \nmid N \Rightarrow \Phi_k(2) \nmid 2^N - 1$ is false at $k = 6$, because $\Phi_6(2) = \Phi_2(2) = 3$. The descending scan nevertheless succeeds because index 6 consumes this shared factor before index 2 is visited. A base-two primitive-divisor theorem, the exceptional reassignment, and the identity $\Phi_n(2^n) = \Phi_{n^2}(2)$ are all proved internally before the final evaluator theorem.

Listing 9: The reported A070526 artifact and its universal Lean theorem.

```
def reportedA070526 : FrozenCandidate where
  programSha256 :=
    "fee6555f282bf4b068dfdc305e404b04b72698a754aa4daed754ed1aaf7e8717"
  tokens := [10,11,7,10,11,6,10,8,11,15,11,1,10,10,3,3,
    11,1,9,10,14,10,13,2,10,3,10,5,1,9]
  program := P.loop
    (P.loop2
      (P.cond (P.modu P.X P.Y) (P.divi P.X P.Y) P.X)
      (P.pop P.Y) P.Y
      (P.push (P.loop (P.addi P.o (P.addi P.X P.X)) P.Y P.o) P.X)
      P.X)
    (P.mult (P.addi P.tw P.X) P.X) P.o
  recognized := rfl

theorem reportedA070526_correct :
    CandidateRealizes CyclotomicSieve.specA070526 reportedA070526
```

Full machine-checked statement and proof: `reportedA070526_correct`.

All four adjudications are positive. This should not be read as an estimated correctness rate: they were selected because the programs had already been presented as particularly interesting explanatory examples [3]. Their evidential value lies elsewhere. They show that stack-using and nested-loop outputs of neural-symbolic synthesis can sometimes be connected to recurrence, fixed-point, and algebraic specifications by universal kernel proofs, while producing reusable semantics and number theory rather than four isolated calculations.

What this does and does not establish. The cohort was selected for formalizability, so 33/33 is emphatically not an estimate of the correctness rate among all 984 discoveries; the sequences resting on three or four published terms are exactly those excluded by that selection, and they remain the open question. More subtly, the proofs certify that each program equals *our formalization* of a definition. Whether that formalization faithfully renders the encyclopedia's source text is a separate judgment, and it is not one a kernel can discharge. We therefore keep two links distinct: program $\equiv$ specification is factive; specification $\equiv$ intended sequence is a reviewed translation carrying its own provenance. Because the proof depends on the translation, the two must not be pooled as independent support—an independent semantic review is a genuinely separate source, and only that can move the second link. This dependence boundary is formalized by `translation_proof_have_no_additiveRevisionLicense`.

The adversarial frontier. The positive cohort does not settle the weakest-evidence part of the catalogue. A candidate-blind registry now locks thirty-three definitions selected from that frontier, with distinct names checked in the kernel (`registry_names_nodup`). It deliberately contains no candidate-realization claims: programs remain hidden from the specification phase. At the registry snapshot no neural discovery had been kernel-refuted; the post-seal adjudication of Section 11.2 has since produced refutations by independent executable replay, which is a strictly weaker evidence grade than a kernel-checked counterexample. The kernel-grade adversarial result must still be reported as a three-way outcome—proof, checked counterexample, or unresolved proof obligation—and the executable adjudication below neither anticipates nor replaces it.

Refutation is generative. A failed adjudication is not a discarded result. A formalized definition is a certified generator: it produces additional terms beyond the published window, with proof, turning a weak finite match into a targeted counterexample and a localized repair obligation—the first index at which the program and the specification provably differ. Ten of our 984 also cover entries the encyclopedia marks `dead`: nine are duplicate identifiers, which are redirected to canonical entries, while one is an erroneous historical version and must not be counted as a current target. Neither case is evidence that a candidate computes the intended canonical sequence. Together these convert the discovery ledger from a count into a graded evidential structure: finite term agreement, keyword and reference-program corroboration, reviewed formalization, and finally factive proof or factive counterexample.

10 From leakage to conjecture

Near-equivalence is dual to conjecture: what threatens a benchmark as contamination is, read the other way, a discovered near-identity. The sharpest instance is the pairing of the *fastest* and the *smallest* candidate covering an entry. These minimize opposite terms of Levin's Kt complexity, $Kt(x) = \min_p |p| + \log t(p)$ [6]: program length and running time. Two unlike programs converging on the same observed terms motivate an equivalence conjecture, but they provide independent evidence only when their search lineages are source-disjoint. Otherwise their common training corpus or ancestral discoveries remain shared causes and additive revision is not licensed. Even with independent provenance, finite agreement is corroboration rather than entailment. *Proving* the two programs equivalent on all n converts finite-prefix agreement into total agreement; the induction predicates such proofs require are exactly what recent conjecturing systems learn to generate [2]. Our verified layer offers to discharge them as kernel-checked theorems rather than trusted external calls. The interesting conjectures live where extensional similarity is high but *syntactic* intensional similarity is low—two unlike programs apparently computing one sequence—and shared formally proved properties become lemmas toward the identity proof.

11 Status and roadmap

The verified DSL semantics, recognizer, big-step modalities, and the three central obligations are machine-checked and axiom-clean. The synthesizer's data pipeline, exact split gate, and the WM-PLN extensional and intensional layers are implemented and measured over the current corpus, with the numbers reported above; the self-learning loop of Section 8 compounds under three registered replay settings across three worlds. Thirty-three frozen discoveries now additionally have full-domain correctness proofs against pinned formal specifications. Near-term work is to (i) independently review the 33 source-to-Lean translations; (ii) adjudicate a cohort selected from the

shortest, weakest-evidence entries, where counterexamples are most likely; (iii) prove the Python abstract interpreter sound against the Lean evaluator before using its conclusions for hard pruning; (iv) bind the Lean canonical policy to exported operator-table metadata; and (v) retain the fastest and smallest program per sequence for provenance-aware equivalence conjectures. Four additional programs reported by Urban and Gauthier's collaboration now have full-domain correctness proofs, including stack-based recurrence and fixed-point programs and a nested cyclotomic sieve [3]. The unifying aim is an OEIS synthesis loop whose catalogue coverage, source interpretation, and full-domain program claims are explicitly distinguished and independently auditable.

A subsequent branch[1] ran three further discovery rounds over the cumulative pool: fifteen cells per round (three carriers, including the new unified carrier, crossed with five update rules, including a new multi-site error-coordinate rule and an independent-BP opportunity control), each rule starting from identical generation-eight parent bytes with predictive coding active from the branch's first update. Round three was prospectively fixed as its terminal resource boundary before results were available; the counts below enter this paper only after the round-three checker seal and an independent longitudinal recount.

This stopping decision is not evidence that the discovery process saturated. The formal boundary is sharper: every finite observed yield prefix has both an eventually-zero continuation and a continuation with positive yield forever, so no classifier of that prefix can identify saturation uniformly (`not_correct_on_zeroTail_and_oneTail`). Continuation is instead a resource decision. Under an explicit positive cost per computation, the expected useful computation of any certified metalevel policy is bounded by its value of perfect information divided by that cost (`expectedComputations_le_valueOfPerfectInformation_div`). Round three therefore closes the branch at its declared resource boundary; any later round would be a new experiment with a new preregistration, not completion of a scientifically privileged number.

11.1 Recorded outcome of the mis-specified branch

The independent recount covers the three branch rounds (pool indices 11–13), 15 cells per round. Cumulative verified union: 747 previously absent A-numbers (BP family 673, PC family 452, overlap 378; Jaccard $378/747 \approx 0.506$; PC training work 6.849× BP).

pool idx	known before	new union	train h	new/train h	pipeline h	new/pipeline h
11	6334	386	20.783	18.573	21.031	18.354
12	6735	225	20.619	10.912	20.855	10.789
13	7081	136	24.217	5.616	24.518	5.547

pool idx	BP	PC	overlap	BP-only	PC-only	PC/BP work
11	353	231	198	155	33	6.968
12	207	134	116	91	18	6.884
13	113	87	64	49	23	6.721

[1] A botched experiment that nonetheless adds data. The branch was intended as a from-scratch comparison of update rules; it was mis-specified and executed as a continuation from generation-eight weights over the generation-ten single-world pool, in one world, under the unchanged registered training protocol. Its verified discoveries enter the solution pool with full provenance. Its update-rule comparisons are recorded for completeness only: they are portfolio observations on a BP-built stack and are not evidence on the from-scratch question. The row labels 11–13 below are pool-accounting indices used by the recording infrastructure, not scientific generation numbers; the branch is parallel to, not a successor of, the generation-nine/ten factorial.

The final round's repair frontier of 132 entries resolved as 17 native rediscoveries, 0 checker-identity rediscoveries, and 115 still absent. The blind cohort (33 sequences) accumulated 0 reviewed refutation edges during the branch; 7 reviews remain pending (the post-seal adjudication of Section 11.2, a distinct cohort, produced the first refutation edges immediately after the seal). The analysis and its independent recomputation have SHA-256 identifiers `3d79d72cfe3cf2ce280e89a3cf1ab3fd5290bf4bd42d98ba0a316a2b69af349b` and `d9204b84f5f17b9205b6ec6aaaca3e8ee2b1b1f7e6534bcb3fe6cdf5e2c04818`.

11.2 Post-seal candidate-blind adjudication

Only after the independently verified branch-terminal seal did we expose the synthesized programs for the frozen adversarial cohort (a cohort distinct from the thirty-three positively adjudicated sequences of Section 9). The reveal contains 46 candidate–sequence edges over 33 sequences. Independent replay against the source prefix and the probes fixed before reveal refutes 11 edges; 7 sequences have at least one refuted edge, and all revealed edges are refuted for 6 sequences. These refutations carry the evidence grade of independent executable recomputation: each is a replay of the candidate program against the pinned source prefix and a probe frozen before reveal, not a kernel-checked counterexample. The kernel-grade three-way adjudication promised at the registry declaration—proof, checked counterexample, or unresolved proof obligation—remains pending for this cohort.

outcome	edges
refuted by a frozen probe	11
passes every frozen probe	9
no frozen continuation probe	14
evaluator resource boundary	12
total	46

The refutations share a single shape. In every refuted edge the frozen probe expects the published sequence to *end*—the entry is finite, so the correct behaviour past its last term is to emit no term—and the program emits an integer instead. Not one refutation is a value disagreement inside the published range. The concentration is equally sharp at the probe-class level: the cohort carries fifteen sequences with frozen generated-value probes, seven with frozen finite-boundary probes, and eleven whose probes await definition derivation; all refuted edges come from the seven finite-boundary sequences (seven of seven refuted), while none of the fifteen value-probed sequences was refuted. The frontier failure mode exposed by this adjudication is therefore not wrong arithmetic but unbounded continuation: programs that extend finite sequences past their end.

OEIS entry	program hash prefix	outcome	first decisive position
A005018	4dc8d3a746fc	REFUTED_BY_FROZEN_PROBE	6
A005018	5b33afae69f1	REFUTED_BY_FROZEN_PROBE	6
A005018	be0c20f99fd7	REFUTED_BY_FROZEN_PROBE	6
A018351	1a1d50e3ea02	REFUTED_BY_FROZEN_PROBE	6
A018351	283049a935fb	REFUTED_BY_FROZEN_PROBE	6
A018351	298d3adfff58	REFUTED_BY_FROZEN_PROBE	6
A185092	1e0e2fd9be05	REFUTED_BY_FROZEN_PROBE	6
A323383	0747df938493	REFUTED_BY_FROZEN_PROBE	7
A005776	275ae516ef5d	REFUTED_BY_FROZEN_PROBE	8
A092596	2831d99fdd3c	REFUTED_BY_FROZEN_PROBE	10

OEIS entry	program hash prefix	outcome	first decisive position
A285198	f4e0b0a41476	REFUTED_BY_FROZEN_PROBE	10

The remaining outcomes must not be collapsed into "correct." Exactly 9 edges pass every available frozen probe—each of these cleared sixteen frozen continuation values beyond the published prefix—while 14 have no frozen continuation probe and 12 encounter an evaluator resource boundary. Passing finite probes is corroboration rather than a universal proof; absence of a probe is an open specification obligation; and resource exhaustion is explicitly not counted as a refutation. The adjudication and its independent replay have SHA-256 identifiers 9765e93e318e4f2df998fde24e0f1d35c5eac5138f2d8d51819dbb31d2cfb4c9 and 8f7a8d8ba832d2ae8e146102e870adc5c05ef8dde8424f5724bae953ade5c61e.

References

[1] T. Gauthier and J. Urban. *Learning Program Synthesis for Integer Sequences from Scratch*. AAAI 2023; arXiv:2202.11908.

[2] T. Gauthier and J. Urban. *Learning Conjecturing from Scratch*. arXiv:2503.01389, 2025.

[3] J. Urban. *Alien Codes and Their Automated and Human Explanations*. Joint work with T. Gauthier, M. Olšák, and T. Hales. Presentation at SCML'26, Linz, July 7, 2026. https://josefurban.eu/slides/scml26oeis.pdf.

[4] OEIS Foundation Inc. *The On-Line Encyclopedia of Integer Sequences* and the `oeisdata` repository. https://oeis.org. Data snapshot: https://github.com/oeis/oeisdata/tree/a6e0f22854cc1c307da428e9d6295093781df7fa. OEIS content is licensed CC BY-SA 4.0.

[5] P. Cousot and R. Cousot. *Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints*. POPL 1977. https://doi.org/10.1145/512950.512973.

[6] L. A. Levin. *Universal Sequential Search Problems*. Problems of Information Transmission, 9(3):265–266, 1973. https://www.mathnet.ru/eng/ppi914.

[7] B. Goertzel, M. Iklé, I. Goertzel, and A. Heljakka. *Probabilistic Logic Networks*. Springer, 2009. https://doi.org/10.1007/978-0-387-76872-4.